```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

apps = pd.read_csv("/content/googleplaystore.csv")
reviews = pd.read_csv("/content/googleplaystore_user_reviews.csv")

print(apps.shape)
print(reviews.shape)

display(apps.head())
display(reviews.head())
```

```
(10841, 13)
(64295, 5)
```


```
{"summary":"{\n  \"name\": \"display(reviews\",\n  \"rows\": 5,\n
\"fields\": [\n    {\n      \"column\": \"App\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n        \"samples\": [\n
\"Coloring book moana\",\n          \"Pixel Draw - Number Art Coloring
Book\",\n          \"U Launcher Lite \\u2013 FREE Live Cool Themes,
Hide Apps\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Category\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 1,\n        \"samples\":
[\n          \"ART_AND_DESIGN\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Rating\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.31622776601683805,\n        \"min\": 3.9,\n        \"max\": 4.7,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          3.9\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Reviews\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          \"967\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Size\",\n      \"properties\":
{\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n
\"samples\": [\n          \"14M\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Installs\",\n      \"properties\":
{\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n
\"samples\": [\n          \"500,000+\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Type\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 1,\n
\"samples\": [\n          \"Free\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
```

```
n    },\n    {\n      \"column\": \"Price\",\n      \"properties\": {\
n        \"dtype\": \"category\",\n        \"num_unique_values\": 1,\n
\"samples\": [\n          \"0\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Content Rating\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"Teen\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Genres\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n          \"Art &
Design\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Last Updated\",\n      \"properties\": {\n        \"dtype\":
\"object\",\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          \"January 15, 2018\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Current Ver\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n        \"samples\": [\n
\"2.0.0\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Android Ver\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"4.0.3 and up\"\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe"}

{"summary":"{\n  \"name\": \"display(reviews\",\n  \"rows\": 5,\n
\"fields\": [\n    {\n      \"column\": \"App\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 1,\n        \"samples\": [\n          \"10 Best
Foods for You\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Translated_Review\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 4,\n        \"samples\":
[\n          \"This help eating healthy exercise regular basis\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Sentiment\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 1,\n        \"samples\": [\n
\"Positive\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Sentiment_Polarity\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.3944933459514875,\n        \"min\":
0.25,\n        \"max\": 1.0,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          1.0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Sentiment_Subjectivity\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.2747617535305011,\n
```

```
\"min\": 0.2884615384615384,\n        \"max\": 0.875,\n
\"num_unique_values\": 4,\n        \"samples\": [\n
0.2884615384615384\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```

```
apps.info()
apps.describe(include="all")
apps.isnull().sum()
apps.duplicated().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10841 entries, 0 to 10840
Data columns (total 13 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   App             10841 non-null  object 
 1   Category        10841 non-null  object 
 2   Rating          9367 non-null   float64
 3   Reviews         10841 non-null  object 
 4   Size            10841 non-null  object 
 5   Installs        10841 non-null  object 
 6   Type            10840 non-null  object 
 7   Price           10841 non-null  object 
 8   Content Rating  10840 non-null  object 
 9   Genres          10841 non-null  object 
 10  Last Updated    10841 non-null  object 
 11  Current Ver     10833 non-null  object 
 12  Android Ver     10838 non-null  object 
dtypes: float64(1), object(12)
memory usage: 1.1+ MB
```

```
np.int64(483)
```

```
apps["Reviews"] = pd.to_numeric(apps["Reviews"], errors="coerce")
apps["Installs"] = (
    apps["Installs"]
    .str.replace(",", "", regex=False)
    .str.replace("+", "", regex=False)
    .str.replace("Free", "0", regex=False)
    .astype(float)
)

apps["Price"] = (
    apps["Price"]
    .str.replace("$", "", regex=False)
    .apply(pd.to_numeric, errors="coerce")
)

def convert_size(value):
    if value == "Varies with device":
        return np.nan
```

```
    if isinstance(value, str):
        if "M" in value:
            return float(value.replace("M", ""))
        elif "k" in value:
            return float(value.replace("k", "")) / 1024

    return np.nan

apps["Size_MB"] = apps["Size"].apply(convert_size)

apps.duplicated().sum()
```

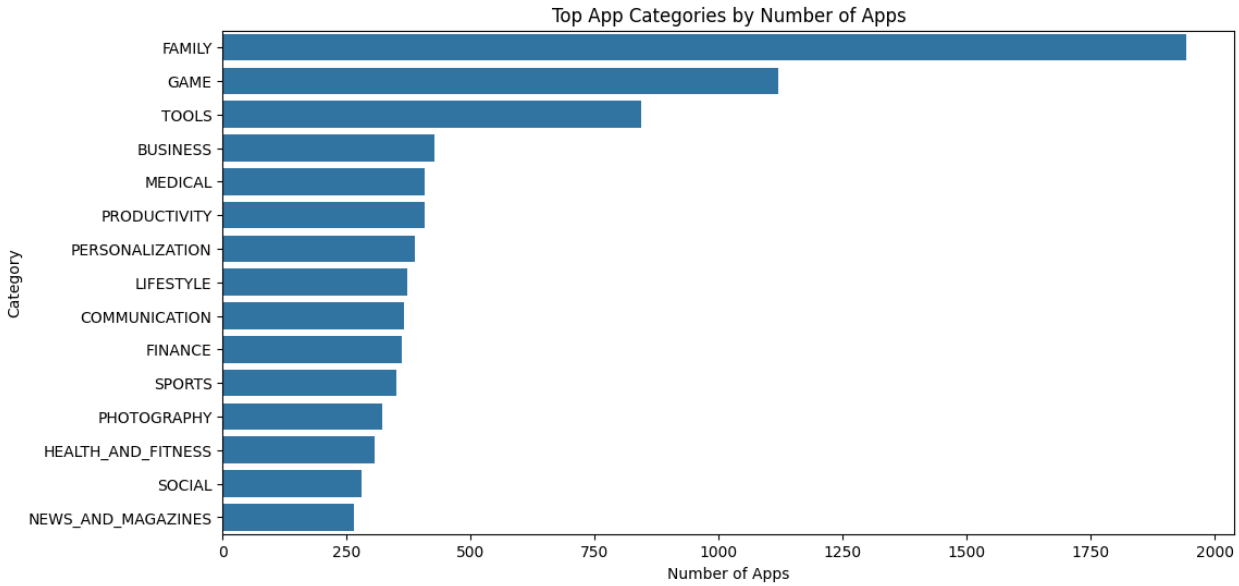
```
np.int64(483)
```

```
apps = apps.drop_duplicates()

apps.isnull().sum().sort_values(ascending=False)
```

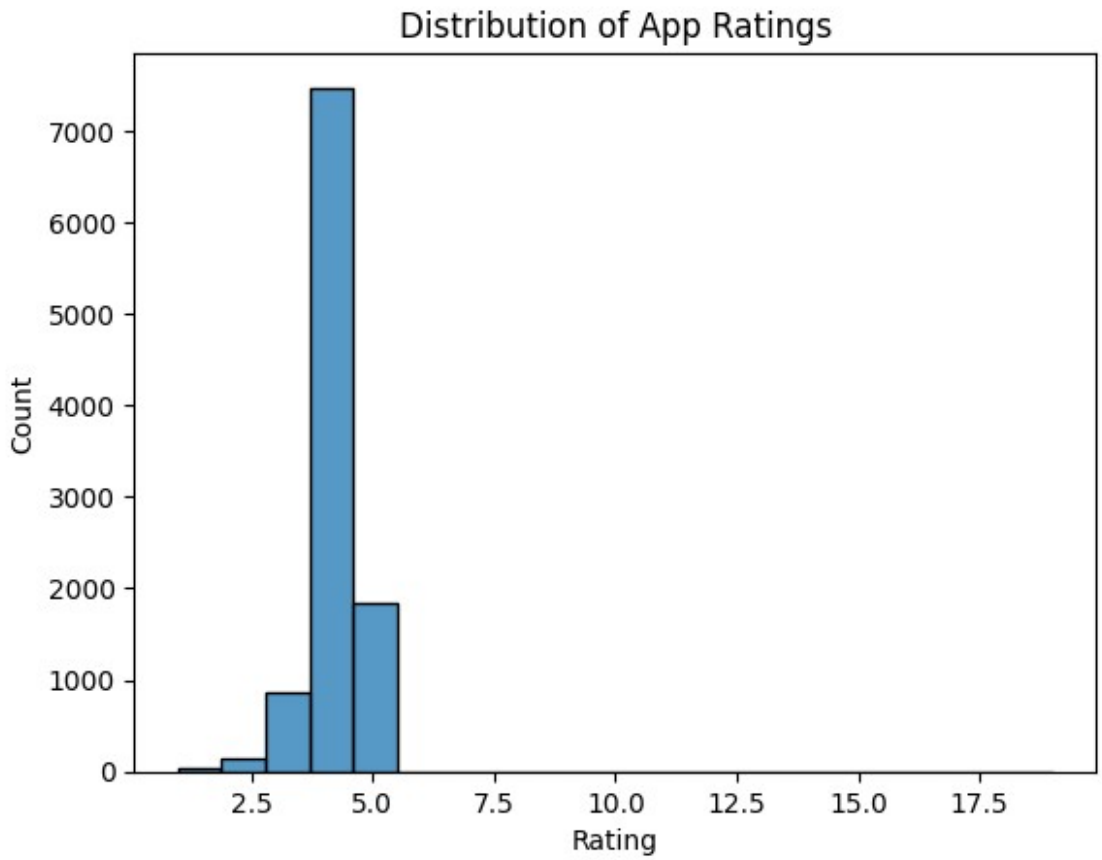
```
Size_MB           1527
Rating            1465
Current Ver          8
Android Ver          3
Type                 1
Reviews              1
Price                1
Content Rating       1
App                  0
Category             0
Installs             0
Size                 0
Genres               0
Last Updated         0
dtype: int64
```

```
apps["Size_MB"] = apps["Size_MB"].fillna(0)
apps["Reviews"] = apps["Reviews"].fillna(0)
apps["Price"] = apps["Price"].fillna(0)

apps["Rating"] = apps["Rating"].fillna(apps["Rating"].median())

category_counts = apps["Category"].value_counts()

plt.figure(figsize=(12,6))
sns.barplot(
    x=category_counts.values[:15],
    y=category_counts.index[:15]
)
plt.title("Top App Categories by Number of Apps")
plt.xlabel("Number of Apps")
```

```
plt.ylabel("Category")
plt.show()
```




```
sns.histplot(apps["Rating"].dropna(), bins=20)
plt.title("Distribution of App Ratings")
plt.show()
```

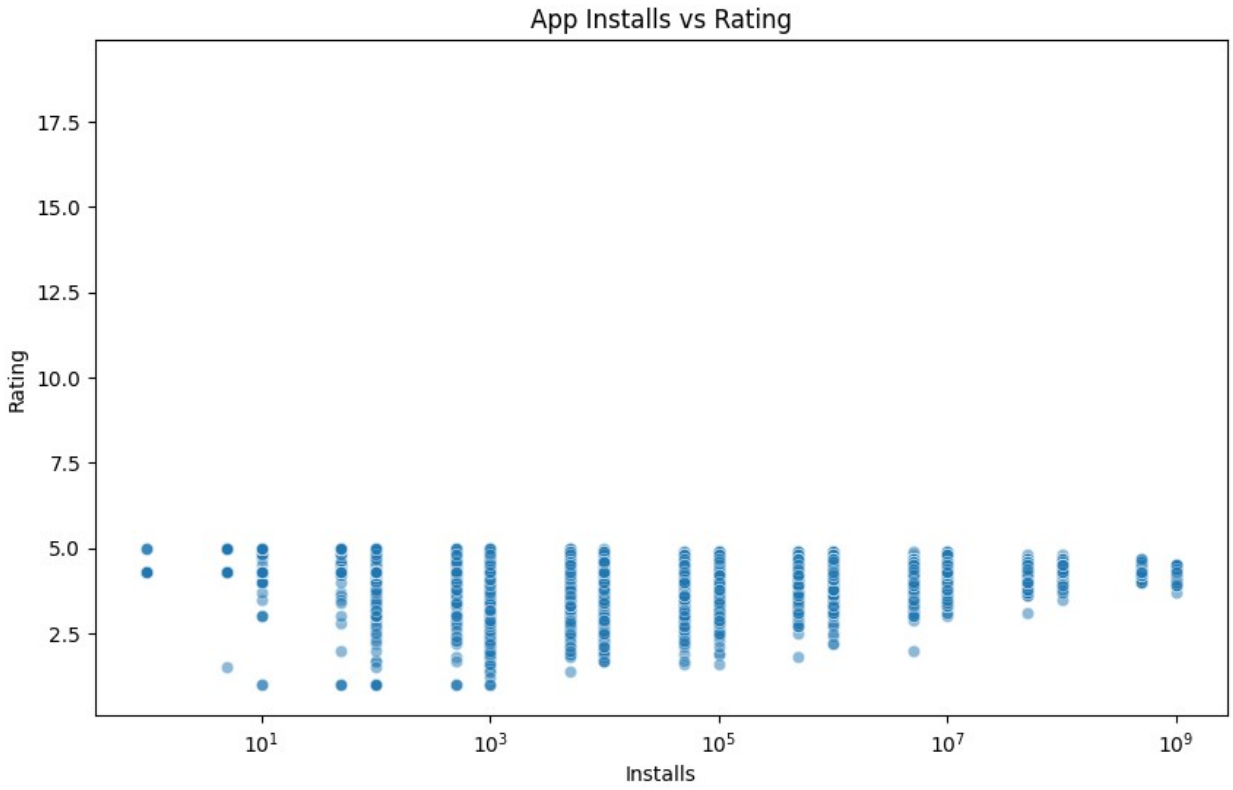



```
category_rating = (
    apps.groupby("Category")["Rating"]
    .mean()
    .sort_values(ascending=False)
)

display(category_rating)
```

```
Category
1.9                    19.000000
EVENTS                  4.395313
EDUCATION               4.375385
ART_AND_DESIGN          4.355385
BOOKS_AND_REFERENCE     4.336522
PERSONALIZATION         4.327062
PARENTING               4.300000
BEAUTY                  4.283019
GAME                    4.282070
HEALTH_AND_FITNESS      4.266993
SOCIAL                  4.260714
SHOPPING                4.256250
WEATHER                 4.248780
SPORTS                  4.239031
```

```
PRODUCTIVITY              4.219410
MEDICAL                   4.212990
LIBRARIES_AND_DEMO        4.207059
AUTO_AND_VEHICLES         4.205882
FAMILY                    4.203757
PHOTOGRAPHY               4.189441
HOUSE_AND_HOME            4.185000
FOOD_AND_DRINK            4.183871
COMMUNICATION             4.175410
BUSINESS                  4.175176
NEWS_AND_MAGAZINES        4.160985
COMICS                    4.160000
FINANCE                   4.148056
ENTERTAINMENT             4.136036
LIFESTYLE                 4.133244
TRAVEL_AND_LOCAL          4.121941
VIDEO_PLAYERS             4.084000
TOOLS                     4.080071
MAPS_AND_NAVIGATION       4.075182
DATING                    4.033673
Name: Rating, dtype: float64
```

```
category_stats = apps.groupby("Category").agg(
    avg_rating=("Rating", "mean"),
    app_count=("App", "count")
)

category_stats = category_stats[
    category_stats["app_count"] >= 10
].sort_values("avg_rating", ascending=False)

plt.figure(figsize=(10,6))

sns.scatterplot(
    data=apps,
    x="Installs",
    y="Rating",
    alpha=0.5
)

plt.xscale("log")
plt.title("App Installs vs Rating")
plt.show()
```



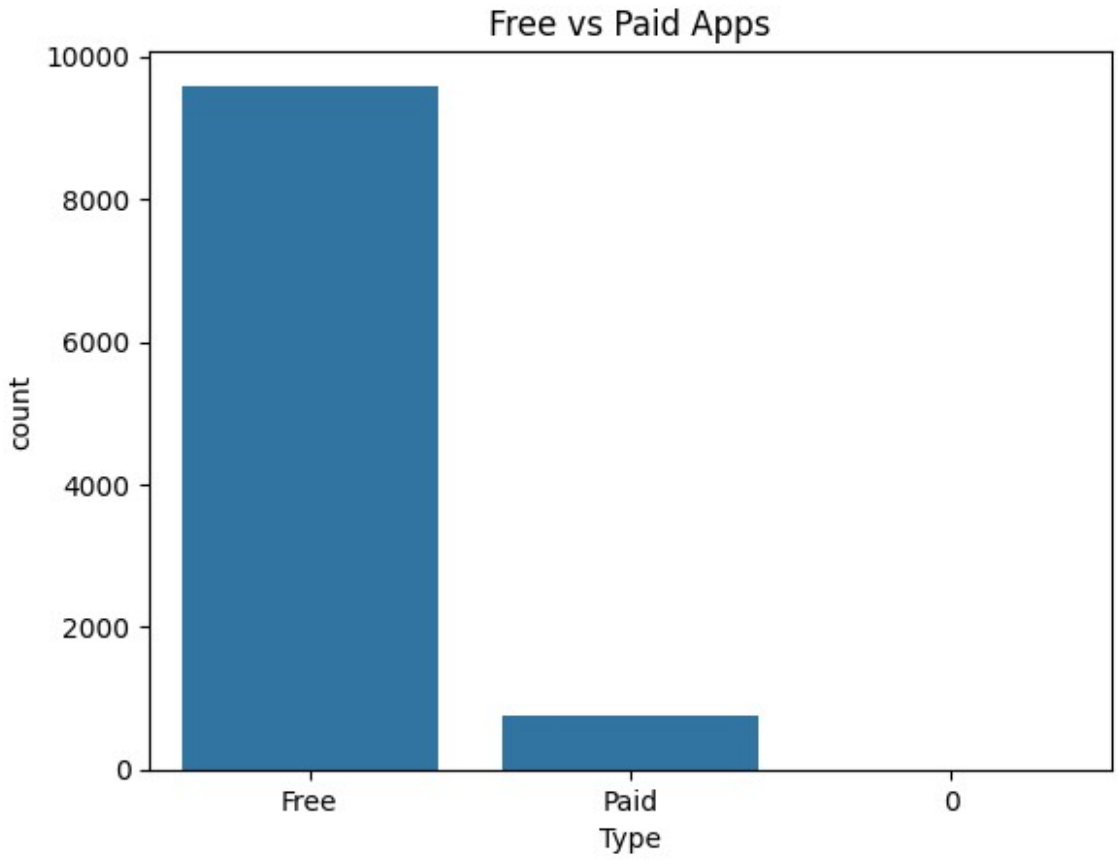

```
apps[["Installs", "Rating"]].corr()
```


```
{"summary":"{\n  \"name\": \"apps[[\\\"Installs\\\",
\\\"Rating\\\"]]\",\n  \"rows\": 2,\n  \"fields\": [\n    {\n
\"column\": \"Installs\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.6770914180586078,\n        \"min\":
0.04244813361508562,\n        \"max\": 1.0,\n
\"num_unique_values\": 2,\n        \"samples\": [\n
0.04244813361508562,\n          1.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Rating\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.6770914180586078,\n        \"min\": 0.04244813361508562,\n
\"max\": 1.0,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          1.0,\n          0.04244813361508562\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```


```
apps["Type"].value_counts()
```

```
Type
Free    9591
Paid     765
0          1
Name: count, dtype: int64
```

```
sns.countplot(data=apps, x="Type")
plt.title("Free vs Paid Apps")
plt.show()
```




```
apps.groupby("Type")["Rating"].mean()

Type
0       19.000000
Free     4.198509
Paid     4.269150
Name: Rating, dtype: float64

apps["Estimated_Revenue"] = (
    apps["Installs"] * apps["Price"]
)

revenue_by_category = (
    apps.groupby("Category")["Estimated_Revenue"]
    .sum()
    .sort_values(ascending=False)
)

reviews.head()
```

{"summary":"{\n \"name\": \"reviews\",\n \"rows\": 64295,\n \"fields\": [\n   {\n   \"column\": \"App\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 1074,\n   \"samples\": [\n   \"Daily Yoga - Yoga Fitness Plans\",\n   \"Calorie Counter - MyNetDiary\",\n   \"Bubble Shooter Genies\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Translated_Review\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 27994,\n   \"samples\": [\n   \"It trick I enjoy I always it.\",\n   \"I wish features..\",\n   \"Nice It's nice , I like it. But fraction calculation number seems overlapping. I hope next update fix it. And unit conversion also please put swapping button easily swap unit i.e. cm km km cm quickly.\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Sentiment\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 3,\n   \"samples\": [\n   \"Positive\",\n   \"Neutral\",\n   \"Negative\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Sentiment_Polarity\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.35130098219622796,\n   \"min\": -1.0,\n   \"max\": 1.0,\n   \"num_unique_values\": 5410,\n   \"samples\": [\n   -0.2005892255892255,\n   -0.244047619047619,\n   -0.6499999999999999\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Sentiment_Subjectivity\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.25994901411056426,\n   \"min\": 0.0,\n   \"max\": 1.0,\n   \"num_unique_values\": 4474,\n   \"samples\": [\n   0.3898484848484849,\n   0.530909090909091,\n   0.7811447811447811\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"reviews"}

```
reviews["Sentiment"].value_counts()
```

```
Sentiment
Positive    23998
Negative     8271
Neutral      5163
Name: count, dtype: int64
```

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import nltk
nltk.download('vader_lexicon')
sia = SentimentIntensityAnalyzer()

reviews["compound"] = reviews["Translated_Review"].fillna("").apply(
```

```
    lambda x: sia.polarity_scores(x)["compound"]
)
```

```
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
```

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import nltk
nltk.download('vader_lexicon')
sia = SentimentIntensityAnalyzer()

reviews["compound"] = reviews["Translated_Review"].fillna("").apply(
    lambda x: sia.polarity_scores(x)["compound"]
)

def sentiment_label(score):
    if score >= 0.05:
        return "Positive"
    elif score <= -0.05:
        return "Negative"
    else:
        return "Neutral"

reviews["Predicted_Sentiment"] =
reviews["compound"].apply(sentiment_label)
```

```
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!
```

```
merged = apps.merge(
    reviews,
    on="App",
    how="left"
)

sentiment_category = pd.crosstab(
    merged["Category"],
    merged["Predicted_Sentiment"],
    normalize="index"
) * 100

import plotly.express as px

fig = px.scatter(
    apps,
    x="Reviews",
    y="Rating",
    size="Installs",
    color="Type",
    hover_name="App",
    log_x=True,
    title="App Rating, Reviews and Install Analysis"
```

```
)

fig.show()
```

Key Business Insights

1. The Family category has the highest app concentration, indicating strong competition and market saturation.

2. Free applications dominate the Play Store dataset, suggesting that developers commonly rely on advertising, subscriptions, or in-app purchases rather than upfront pricing.

3. User sentiment varies substantially across categories, highlighting areas where developers can improve user experience.

4. App popularity and rating do not necessarily move together, indicating that high installation volume does not automatically imply higher user satisfaction.

5. Revenue estimates show that a small number of paid applications can account for a disproportionately large share of estimated gross revenue.